

number generators typically make use of a discrete event simulation model.

Another point to be discussed is the difference between simulation and emulation. The latter is the process by which one system is made to behave like another system. This is different from the process of simulation. An emulator can be used to carry out real tasks as a substitute for the original device or system, whereas a simulator is used mainly as a model for analysis. For example, QEMU is a hosted virtual machine monitor. It emulates real CPUs and can be used to carry out arithmetic operations just like a normal CPU, whereas an automobile simulator is simply used for training and not for the actual purpose of transportation. I hope that example makes the difference between an emulator and a simulator clear.

An introduction to network simulation

Network simulation is the process by which a computer network is modelled by identifying, analysing and quantifying the interaction between various network devices and software. Mathematical modelling is used to study the behaviour of a computer network rather than using actual data. There are many advantages of doing this. Network behaviour under extreme conditions can be analysed very easily using a simulator. For example, consider a node (network simulation jargon to refer to a computer system) with mobility in a wireless network. We can very easily study the behaviour of a node having high velocity, using a simulator. In a physical network, achieving high velocity for mobile nodes is a very difficult task, and varying the velocity of mobile nodes for analysis purposes is time consuming and difficult.

Another situation where simulation becomes the natural choice is when the number of nodes in the network is very high. For example, if you want to study the working of a network with, say, a 1000 nodes, then simulation is the ideal choice, at least till you are sure about the results. Simulators are also suitable for network analysis if you have a limited budget, limited time, or you are not sure about the success of your proposed method or protocol.

While the previous paragraph lists out a few scenarios where network simulation is warranted, don't assume that network simulation is an easy task. Simulating networks is difficult and developing a network simulator is an almost impossible task. Very often, I have lamented that it is far better to set up a network in hardware than use these simulators (at least, I used to in my younger days). To crown it all, sometimes the data obtained from the actual physical network may not corroborate the results obtained from the simulator. Having discussed all the many negative aspects of network simulators, I still believe they are excellent tools and all researchers should arm themselves with these potent tools.

Even though the ns class of simulators is the most widely used, there are other options in this field. There are QualNet, NetSim, OMNeT++, etc, which also are useful network simulators. If there are so many alternative network simulators,

why is the ns class the most pervasive? QualNet and NetSim are proprietary tools and fail to address the important problem of reducing research costs, whereas the ns class of simulators is free and open source software that can be freely downloaded from the Internet. OMNeT++ is free software but it is not a complete simulator; rather, it is a simulation framework. So you have to develop your own modules before simulating the network, and this may not interest many potential users. So considering all these factors, I believe ns is the best tool available for a budding researcher in the area of computer networks.

A brief history of ns

The story of ns begins with a simulator called REAL, which was developed in 1989. This simulator has heavily influenced the first version of ns (called ns-1), which was developed in 1995. ns-1 was mostly a derivative work of the REAL simulator. The creation of ns-1 was initiated by the Lawrence Berkeley National Laboratory (LBNL), and later received large scale contributions from Sun Microsystems and Carnegie Mellon University. The core of the system was developed in C++ and simulation scripts were written using Tcl, a scripting language. Tcl is used as a binding language for generating simulation scripts, and saves a lot of time by avoiding unnecessary recompilation of the C++ core code every time a minor modification is made in the simulation scenario. This bi-language system was really beneficial in the earlier days when processing power was very low, but this approach is not necessary nowadays with processors measuring their speeds in gigafLOPS and terafLOPS.

The second version, ns-2, was initially made available for researchers in 1997. It was not very different from ns-1. The core of ns-2 was also developed in C++ with many of the features directly taken from ns-1. One major difference was that in ns-2, the binding language was OTcl rather than Tcl. OTcl is an object oriented dialect of Tcl developed by MIT. ns-2 was instantly adopted by many research communities all over the world, even though there were problems like a steep learning curve and a large number of bugs.

The third version of the network simulator, ns-3, was first released in 2008. The US National Science Foundation (NSF) funded the development of ns-3 by Tom Henderson and team. ns-3 is a free and open source discrete event simulator licensed under the GNU GPLv2 license. The latest version of ns-3 is ns-3.22, which was released in February 2015. Unlike ns-2, developers make sure that there are frequent and regular releases of ns-3. The close relationship between ns-1 and ns-2 has led to the misconception that ns-3 is also not very different from them. The decision to name the simulator ns-3 seems to be an ill-advised one, because a different name would have generated a larger fan base for this elegant software.

The relevance of ns-3

I believe people will let go of ns-2 only if the case for ns-3 is strong and convincing. That is my sole aim now. The